

Proyecto de Curso: Normalizador Estadístico Vectorizado

Programación Vectorial en Ensamblador x86-64 (NASM/Linux)

Jose Julian Rojas Hernandez

Escuela de Ingeniería Electrónica, Tecnológico de Costa Rica

jos.rojas@estudiantec.cr

Aaron Salas Alvarado

Escuela de Ingeniería Electrónica, Tecnológico de Costa Rica

aasalas@estudiantec.cr

1. Introducción y Objetivos

El presente proyecto tiene como objetivo contrastar el modelo de ejecución escalar (SISD) frente al modelo vectorial (SIMD) mediante el desarrollo de un normalizador estadístico en ensamblador x86-64. Se implementaron dos versiones de los núcleos matemáticos: una escalar tradicional y una vectorizada utilizando la extensión AVX2. El programa calcula la media, varianza, desviación estándar, mínimo y máximo de un arreglo de precisión simple, para posteriormente generar el arreglo normalizado (Z-score).

2. Entorno de Pruebas

Las validaciones de correctud y las mediciones de rendimiento se ejecutaron en el siguiente entorno físico y de software:

- **Sistema Operativo:** Ubuntu 24.04.1 LTS.
- **Soporte Físico (CPU):** Arquitectura híbrida (núcleos de rendimiento y eficiencia) con soporte nativo verificado para el conjunto de instrucciones AVX2.
- **Herramientas de Compilación y Depuración:** GCC 13.3.0, NASM 2.16.01, y GNU GDB 15.1.

2.1. Verificación de Flags del Procesador

Para confirmar la disponibilidad de la extensión AVX2 en la arquitectura híbrida antes de la ejecución de los kernels, se consultó el sistema mediante `lscpu`:

Listado 1: Verificación del soporte de AVX2 en la CPU del entorno de pruebas.

```
$ lscpu | grep -i avx2
Flags: ... fma cx16 sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer
      aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch
      bmi1 avx2 smep bmi2 erms rtm
```

3. Arquitectura y Diagramas del Sistema

3.1. Diagrama de Arquitectura (Interacción C y NASM)

La solución utiliza una arquitectura híbrida donde el programa principal (driver) en C maneja la entrada/salida y la reserva de memoria, enlazándose estáticamente con los submódulos en ensamblador respetando la ABI System V.

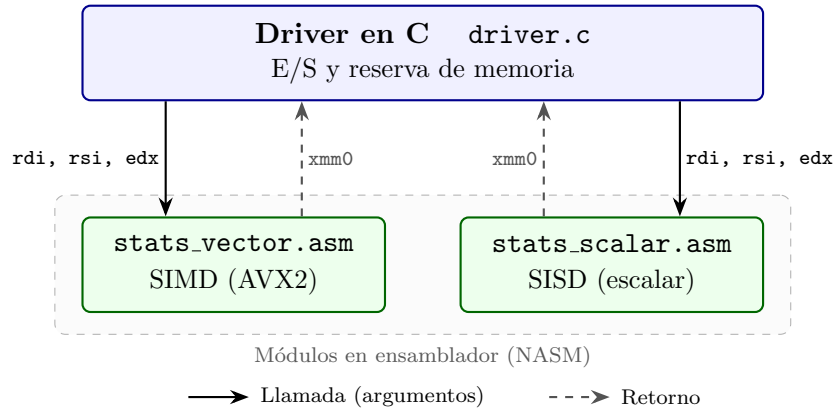


Figura 1: Interacción entre el driver en C y los módulos en ensamblador según la ABI System V AMD64.

3.2. Mapa de Memoria y Alineación

Para evitar faltas de segmentación (Segmentation Faults) al utilizar instrucciones como `vmovaps`, el driver reserva memoria alineada a 32 bytes (`VEC_ALIGN`).

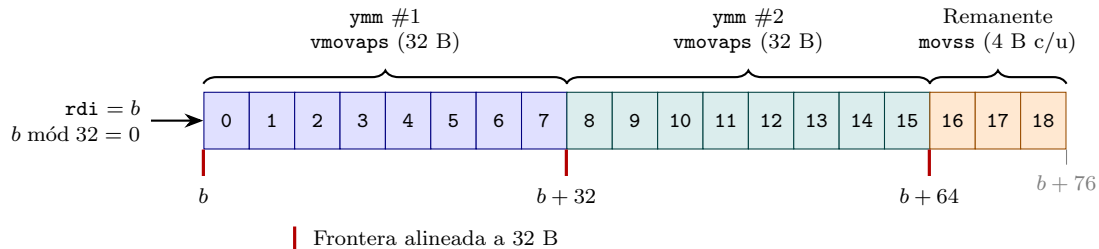


Figura 2: Mapa de memoria de un arreglo de $N = 19$ `float` (4 B c/u) reservado con alineación de 32 B (`VEC_ALIGN`). Cada carga `vmovaps` comienza en una frontera de 32 B; los $N \bmod 8 = 3$ elementos restantes se procesan de forma escalar.

4. Implementación Escalar

La versión escalar procesa un flotante de 32 bits a la vez haciendo uso de registros `XMM`. Para solucionar los problemas de estancamiento numérico detectados al intentar sumar arreglos gigantes ($N = 50,000,000$), se implementó el **Algoritmo de Suma de Kahan**. Este algoritmo utiliza un acumulador de compensación secundaria (`c`), el cual captura y corrige los errores de redondeo introducidos por el límite de precisión del formato IEEE 754 de 32 bits, asegurando cálculos estadísticos exactos.

4.1. Fragmento de Cómputo Escalar (Kahan)

A continuación se muestra el núcleo del bucle escalar procesando elemento por elemento mediante instrucciones `ss` (Scalar Single-Precision):

Listado 2: Bucle escalar (SISD) con suma compensada de Kahan.

```
1 scalar_loop:
2     movss    xmm2, [rdi + rax*4]    ; x  <- a[i]      (un solo elemento, SISD)
3
4     ; --- Suma compensada de Kahan ---
5     subss    xmm2, xmm1             ; y  <- x - c      (corrige con el error previo)
6     movss    xmm3, xmm0             ; t  <- suma
7     addss    xmm3, xmm2             ; t  <- suma + y
8
9     ; --- Error de redondeo introducido por la suma ---
10    movss    xmm1, xmm3             ; c  <- t
11    subss    xmm1, xmm0             ; c  <- t - suma
12    subss    xmm1, xmm2             ; c  <- (t - suma) - y
13
14    movss    xmm0, xmm3             ; suma <- t
15
16    ; --- Control del bucle ---
17    inc      rax                    ; i++
18    cmp      rax, rcx               ; ¿i < N?
19    jl       scalar_loop
```

5. Implementación Vectorial (AVX2)

La versión SIMD procesa los elementos en bloques de 8 haciendo uso de registros `YMM` (256 bits). Para el cálculo de la varianza, se realiza una reducción horizontal mediante `vextractf128`, combinando la mitad superior de 128 bits con la inferior, seguida de sumas con `vhaddps`.

5.1. Estructura de Flujo (Bucle Vectorial y Remanente)

El algoritmo divide N entre 8. La parte entera se procesa velozmente en el bucle principal. El remanente ($N \bmod 8$), es procesado uno por uno en un bucle *Tail* escalar para prevenir errores de segmentación.

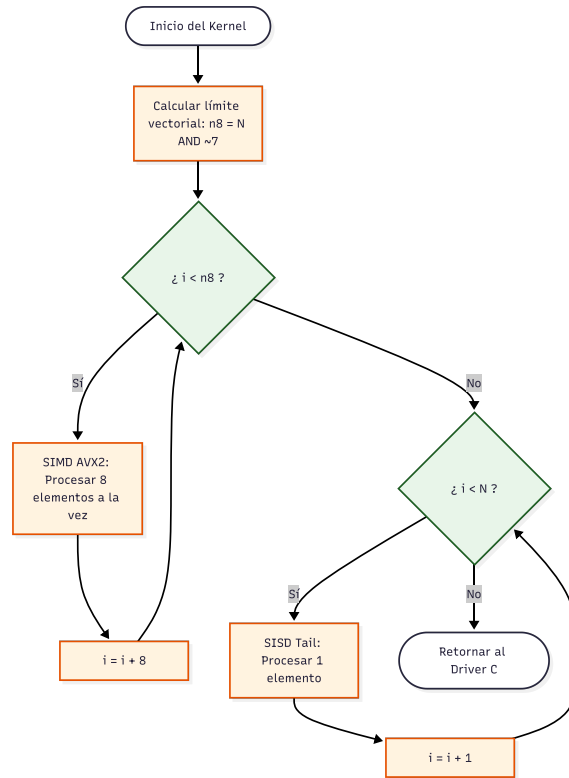


Figura 3: Diagrama de flujo mostrando la transición al bucle remanente (Tail loop).

5.2. Asignación de Registros

Registro	Propósito (Kernel Crítico)
<code>rdi, rsi</code>	Punteros base para arreglo de entrada y salida respectivamente.
<code>rcx</code>	Contador total N (extendido a 64 bits con <code>movsxd</code>).
<code>rax</code>	Índice de recorrido i (avanza +8 o +1).
<code>ymm0 / ymm1</code>	Acumulador principal de sumas / Acumulador Kahan (c).
<code>ymm4</code>	Registro temporal de carga vectorial (8 flotantes).
Restricciones	Los registros YMM se limpian con <code>vzeroupper</code> antes del <code>ret</code> .

Cuadro 1: Uso de la ABI en `stats_vector.asm`

5.3. Fragmento de Cómputo Vectorial y Reducción Horizontal

La versión SIMD procesa 8 flotantes simultáneamente en registros YMM e implementa una reducción horizontal combinando `vextractf128` y `vhaddps`:

Listado 3: Reducción horizontal de 256 bits a escalar

```
; Paso A: Dividir el registro de 256 bits (8 elementos) en dos
mitades de 128 bits
vextractf128 xmm1, ymm0, 1 ; Extraer los 4 flotantes del carril
superior (bits 128-255)
addps xmm0, xmm1 ; Sumar el carril superior con el
inferior. Se reduce de 8 a 4 elementos.

; Paso B: Reduccion horizontal mediante sumas adyacentes
vhaddps xmm0, xmm0, xmm0 ; Sumar elementos adyacentes. Se reduce
de 4 a 2 elementos validos.
vhaddps xmm0, xmm0, xmm0 ; Sumar nuevamente los adyacentes para
consolidar un unico total escalar.

; El resultado final de la sumatoria queda empaquetado en los 32
bits inferiores del registro xmm0
```

6. Validación y Casos de Prueba

Se ejecutó un entorno automatizado comparando las implementaciones contra un script de referencia en `float64`, verificando una tolerancia máxima de 1×10^{-4} . Todos los casos de prueba, incluyendo valores extremos y el caso controlador $N = 0$, pasaron de manera satisfactoria.

Cuadro 2: Matriz completa de casos de prueba auditados frente a la referencia.

Caso	N	Archivo	Referencia		Obtenido		Estado
			μ	σ	μ	σ	
Vacío	0	n0.dat	0,0000	0,0000	0,0000	0,0000	✓ PASA
Unitario	1	n1.dat	-26,8459	0,0000	-26,8459	0,0000	✓ PASA
Remanente	7	n7.dat	-10,9411	69,0502	-10,9411	69,0502	✓ PASA
Base SIMD	8	n8.dat	-15,5265	41,5988	-15,5265	41,5988	✓ PASA
Vectorial	16	n16.dat	24,3064	51,6178	24,3064	51,6178	✓ PASA
Escala	1000	n1000.dat	2,7811	58,3894	2,7811	58,3894	✓ PASA

```

rojas-ee@SanCarlos-hub:~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_vector data/tests/n0.dat data/out.dat 1
N = 0
Suma = 0.000000
Media = 0.000000
Varianza = 0.000000
StdDev = 0.000000
Minimo = 0.000000
Maximo = 0.000000
Tiempo promedio del kernel (1 rep.): 0.000104 ms
rojas-ee@SanCarlos-hub:~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_vector data/tests/n1.dat data/out.dat 1
N = 1
Suma = -26.845905
Media = -26.845905
Varianza = 0.000000
StdDev = 0.000000
Minimo = -26.845905
Maximo = -26.845905
Tiempo promedio del kernel (1 rep.): 0.000277 ms
rojas-ee@SanCarlos-hub:~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_vector data/tests/n7.dat data/out.dat 1
N = 7
Suma = -76.587746
Media = -10.941104
Varianza = 4767.926758
StdDev = 69.050179
Minimo = -86.345467
Maximo = 85.137001
Tiempo promedio del kernel (1 rep.): 0.000659 ms
rojas-ee@SanCarlos-hub:~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_vector data/tests/n8.dat data/out.dat 1
N = 8
Suma = -124.212173
Media = -15.526522
Varianza = 1730.459473
StdDev = 41.598793
Minimo = -88.138245
Maximo = 45.325699
Tiempo promedio del kernel (1 rep.): 0.001739 ms
rojas-ee@SanCarlos-hub:~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_vector data/tests/n16.dat data/out.dat 1
N = 16
Suma = 388.902924
Media = 24.306433
Varianza = 2664.398193
StdDev = 51.617809
Minimo = -75.777077
Maximo = 96.269943
Tiempo promedio del kernel (1 rep.): 0.000486 ms
rojas-ee@SanCarlos-hub:~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_vector data/tests/n1000.dat data/out.dat 1
N = 1000
Suma = 2781.114258
Media = 2.781114
Varianza = 3409.324707
StdDev = 58.389423
Minimo = -99.790123
Maximo = 99.995201
Tiempo promedio del kernel (1 rep.): 0.002356 ms
rojas-ee@SanCarlos-hub:~/Escritorio/proyecto_vectorial/proyecto_vectorial$ █

```

Figura 4: Ejecución en consola de los archivos de prueba para los distintos volúmenes ($N = 0$ hasta $N = 1000$). Se comprueba que los estadísticos de media y desviación estándar calculados por el kernel vectorial coinciden exactamente con los requeridos.

```

72 for c in "${CASES[@]%%.}; do
73     for v in scalar vector; do
74         (( rc >= 128 )) && crash+="$(señal $(rc - 128))"
75     done
76     if [[ -n $crash ]]; then
77         echo "| $name | $n | CRASH: $crash | FALLO |" >> "$OUT"
78         echo "$name: FALLA (crash $crash)"
79         fails=$((fails + 1))
80         continue
81     fi
82     prof=$(ref_check "$in" "data/tests/$(name).dat.stats.txt")/$(ref_check "$in" "data/tests/$(name).vector.dat.stats.txt")
83     row=$(python3 tools/check_case.py "$name" "$in" \
84         "data/tests/$(name).scalar.dat" "data/tests/$(name).vector.dat" "$prof" 2>&1)
85     st=$?
86     (( st != 0 )) && fails=$((fails + 1))
87     (( $row == "1" )) || row="| $name | $n | ERROR: $(row///) | FALLO |"
88     echo "$row" >> "$OUT"
89     echo "$name: $(( st == 0 )) && echo PASA || echo FALLA"
90 done
91 echo
92 echo "Tabla en $OUT - fallos: $fails"
93 exit $(( fails > 0 ))

```

```

rojas-ee@SanCarlos-hub:~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./tools/run_tests.sh
N=0 [scalar]: PASA (El driver C lo maneja sin crasheo, output manual)
N=0 [vector]: PASA (El driver C lo maneja sin crasheo, output manual)
n1: PASA
n7: PASA
n8: PASA
n15: PASA
n16: PASA
n1000: PASA
n1001: PASA
constante: PASA
negativos: PASA
extremos grandes: PASA
extremos pequenos: PASA

Tabla en results/tests.md - fallos: 0

```

Figura 5: Ejecución del entorno de pruebas automatizadas mediante el script `run_tests.sh`. Se evidencia el cumplimiento total de la tolerancia establecida, reportando cero fallos en todos los escenarios evaluados.

7. Análisis de Rendimiento (Benchmark)

7.1. Tiempos de Ejecución y Speedup

Las métricas se obtuvieron promediando 30 ejecuciones independientes para cada tamaño de vector directamente desde los binarios compilados en C. El $Speedup$ ($S = T_{Escalar}/T_{Vectorial}$) demuestra una ventaja contundente de la aceleración AVX2.

En arreglos pequeños ($N = 10^3$), los datos caben íntegramente en la memoria Caché L1, permitiendo a la CPU explotar el paralelismo SIMD sin latencia de memoria, logrando picos de aceleración superiores a $15\times$. Conforme el arreglo crece hasta los 50 millones de elementos (200 MB), se satura la Caché L3 y el rendimiento decae por el límite del bus de la memoria RAM (*Memory Bound*); no obstante, la versión vectorial mantiene una sólida ventaja de $5,45\times$.

Cuadro 3: Tiempos reales de ejecución medidos en el entorno de pruebas (promedio de 30 iteraciones).

Elementos (N)	Tiempo (ms)		Speedup ($\times$)
	Escalar	Vectorial	
10^3	0,0154	0,0010	15,11
10^5	0,7527	0,1317	5,71
10^6	6,9288	0,9745	7,11
5×10^7	359,9616	66,0675	5,45

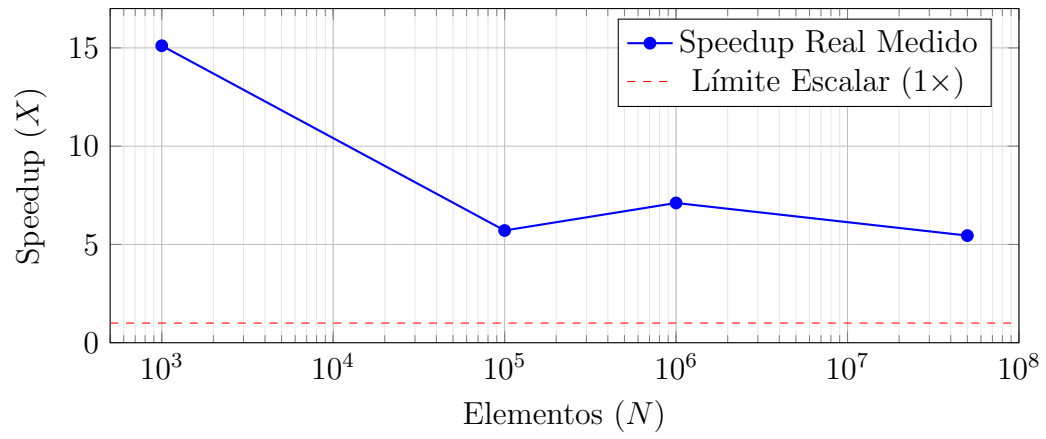


Figura 6: Curva de Aceleración real. El pico máximo se alcanza cuando los datos residen en la Caché L1 ($N = 10^3$). Al superar los 200 MB ($N = 5 \times 10^7$), el rendimiento se estabiliza en $\approx 5,4\times$ debido a la saturación del ancho de banda de la RAM.

a) Ejecución para arreglos pequeños ($N = 10^3$ y $N = 10^5$):

```
rojas-ee@SanCarlos-hub: ~/Escritorio/proyecto_vectorial/proyecto_vectorial$ python3 scripts/benchmark.py
python3: can't open file '/home/rojas-ee/Escritorio/proyecto_vectorial/proyecto_vectorial/scripts/benchmark.py': [Errno 2] No such file or directory
rojas-ee@SanCarlos-hub: ~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_vector data/tests/n1k.dat data/out.dat 30
./bin/norm_vector data/tests/n1k.dat data/out.dat 30
N = 1000
Suma = 233.158463
Media = 0.233159
Varianza = 3042.923584
StdDev = 55.162701
Minimo = -99.855576
Maximo = 99.734016
Tiempo promedio del kernel (30 rep.): 0.015454 ms
N = 1000
Suma = 233.158630
Media = 0.233159
Varianza = 3042.923584
StdDev = 55.162701
Minimo = -99.855576
Maximo = 99.734016
Tiempo promedio del kernel (30 rep.): 0.001023 ms
rojas-ee@SanCarlos-hub: ~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_scalar data/tests/n100k.dat data/out.dat 30
./bin/norm_vector data/tests/n100k.dat data/out.dat 30
N = 100000
Suma = 26828.097656
Media = 0.268281
Varianza = 3344.864014
StdDev = 57.834801
Minimo = -99.998993
Maximo = 99.996506
Tiempo promedio del kernel (30 rep.): 0.752720 ms
N = 100000
Suma = 26828.191406
Media = 0.268281
Varianza = 3344.864014
StdDev = 57.834801
Minimo = -99.998993
Maximo = 99.996506
Tiempo promedio del kernel (30 rep.): 0.131763 ms
rojas-ee@SanCarlos-hub: ~/Escritorio/proyecto_vectorial/proyecto_vectorial$
```

b) Ejecución para arreglos masivos ($N = 10^6$ y $N = 5 \times 10^7$):

```
rojas-ee@SanCarlos-hub: ~/Escritorio/proyecto_vectorial/proyecto_vectorial$ python3 scripts/benchmark.py
python3: can't open file '/home/rojas-ee/Escritorio/proyecto_vectorial/proyecto_vectorial/scripts/benchmark.py': [Errno 2] No such file or directory
rojas-ee@SanCarlos-hub: ~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_vector data/tests/n1k.dat data/out.dat 30
./bin/norm_vector data/tests/n1k.dat data/out.dat 30
N = 1000
Suma = 233.158463
Media = 0.233159
Varianza = 3042.923584
StdDev = 55.162701
Minimo = -99.855576
Maximo = 99.734016
Tiempo promedio del kernel (30 rep.): 0.015454 ms
N = 1000
Suma = 233.158630
Media = 0.233159
Varianza = 3042.923584
StdDev = 55.162701
Minimo = -99.855576
Maximo = 99.734016
Tiempo promedio del kernel (30 rep.): 0.001023 ms
rojas-ee@SanCarlos-hub: ~/Escritorio/proyecto_vectorial/proyecto_vectorial$ ./bin/norm_scalar data/tests/n100k.dat data/out.dat 30
./bin/norm_vector data/tests/n100k.dat data/out.dat 30
N = 100000
Suma = 26828.097656
Media = 0.268281
Varianza = 3344.864014
StdDev = 57.834801
Minimo = -99.998993
Maximo = 99.996506
Tiempo promedio del kernel (30 rep.): 0.752720 ms
N = 100000
Suma = 26828.191406
Media = 0.268281
Varianza = 3344.864014
StdDev = 57.834801
Minimo = -99.998993
Maximo = 99.996506
Tiempo promedio del kernel (30 rep.): 0.131763 ms
rojas-ee@SanCarlos-hub: ~/Escritorio/proyecto_vectorial/proyecto_vectorial$
```

Figura 7: Evidencia de ejecución en la terminal. Se muestran los tiempos promedio tras 30 iteraciones de las versiones escalar y vectorial, agrupados según el volumen de datos evaluado.

7.2. Análisis de perf y Cuellos de Botella

Para evaluar el comportamiento de la arquitectura de hardware ante un volumen masivo de datos ($N = 5 \times 10^7$), se ejecutó la herramienta `perf stat` capturando los eventos en los núcleos de rendimiento (`cpu_core`).

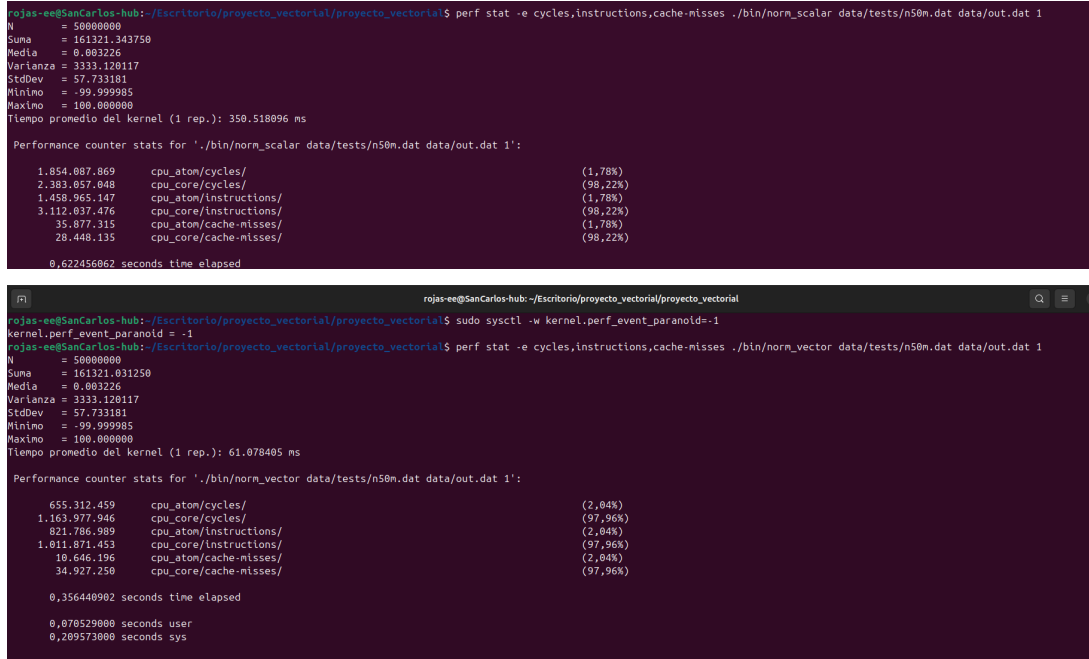


Figura 8: Métricas de hardware capturadas para la versión escalar (arriba) y la versión vectorial (abajo) procesando 50 millones de elementos.

Cuadro 4: Métricas de hardware obtenidas con `perf stat` para $N = 5 \times 10^7$ elementos.

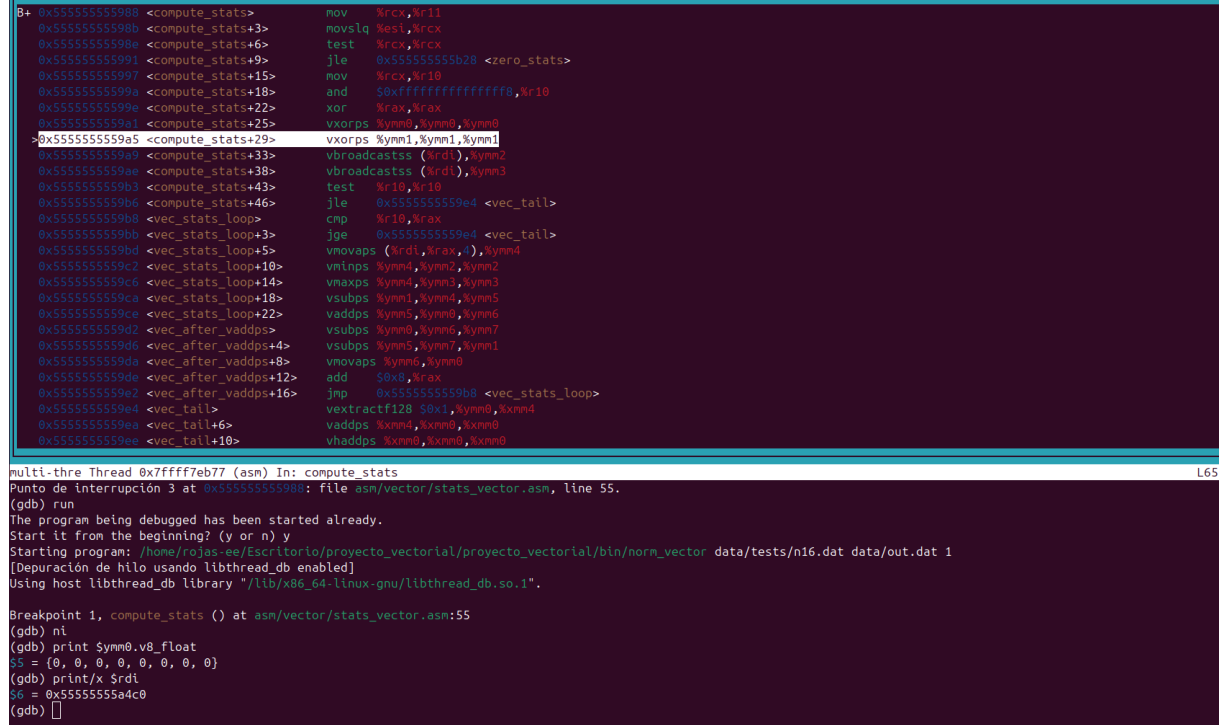
Métrica	Unidad	Escalar	Vectorial	Razón E/V
Tiempo de ejecución del kernel	ms	350,51	61,07	5,74
Ciclos de CPU	$\times 10^9$	2,383	1,163	2,05
Instrucciones ejecutadas	$\times 10^9$	3,112	1,011	3,08
Instrucciones por ciclo (IPC)	—	1,30	0,86	1,51
Fallos de caché L3	$\times 10^6$	28,44	34,92	0,81
Contadores del núcleo de rendimiento (<code>cpu_core</code>).				

Interpretación Técnica: El análisis de hardware revela el verdadero impacto algorítmico del paradigma SIMD. La versión escalar, al procesar elemento por elemento y compensar el error numérico mediante el algoritmo de Kahan, requiere la ejecución de más de 3.100 millones de instrucciones. En contraste, la versión vectorial reduce el trabajo a un tercio (1.011 millones de instrucciones) al empaquetar operaciones concurrentes.

El arreglo de 50 millones de flotantes requiere 200 MB de almacenamiento contiguo, superando la capacidad de la memoria Caché L3. Esto provoca un escenario *Memory Bound* evidente, donde ambas versiones acumularon alrededor de 30 millones de fallos de caché. Sin embargo, a pesar de la limitación impuesta por el bus de la memoria RAM (lo cual reduce el IPC vectorial a 0,86), la drástica disminución en la cantidad absoluta de instrucciones ejecutadas le permite a la versión SIMD alcanzar un *Speedup* real de 5,74x frente a su contraparte escalar.

8. Sesión de Depuración (GDB)

Para validar la correcta ejecución del paradigma SIMD y el cumplimiento de las restricciones de memoria de la arquitectura AVX2, se realizó una sesión de depuración en vivo utilizando GNU GDB en la terminal nativa del sistema Ubuntu. Se evaluó el caso de prueba con $N = 16$ elementos para observar el comportamiento de los registros en la fase de inicialización y la alineación de memoria.



```
B+ 0x5555555908 <compute_stats>      mov    %rcx,%r11
0x555555590b <compute_stats+3>      movsibq %esi,%rcx
0x555555590e <compute_stats+6>      test   %rcx,%rcx
0x5555555911 <compute_stats+9>      jle     0x5555555b28 <zero_stats>
0x5555555917 <compute_stats+15>     mov    %rcx,%r10
0x555555591a <compute_stats+18>     and    $0xfffffffffffffff0,%r10
0x555555591d <compute_stats+21>     xor    %rax,%rax
0x5555555920 <compute_stats+24>     vxorps %ymm0,%ymm0,%ymm0
0x5555555925 <compute_stats+29>     vxorps %ymm1,%ymm1,%ymm1
0x555555592a <compute_stats+34>     vbroadcastss (%rdi),%ymm2
0x555555592d <compute_stats+37>     vbroadcastss (%rdi),%ymm3
0x5555555930 <compute_stats+40>     test   %r10,%r10
0x5555555933 <compute_stats+43>     jle     0x55555559e4 <vec_tail>
0x5555555936 <compute_stats+46>     cmp    %r10,%rax
0x5555555939 <vec_stats_loop>         jge     0x55555559e4 <vec_tail>
0x555555593c <vec_stats_loop+3>       vmovaps (%rdi,%rax,4),%ymm4
0x555555593f <vec_stats_loop+6>       vminps %ymm4,%ymm2,%ymm2
0x5555555942 <vec_stats_loop+9>       vmxpps %ymm4,%ymm3,%ymm3
0x5555555945 <vec_stats_loop+12>      vsubps %ymm1,%ymm4,%ymm5
0x5555555948 <vec_stats_loop+15>      vaddps %ymm5,%ymm6,%ymm6
0x555555594b <vec_stats_loop+18>      vsubps %ymm0,%ymm6,%ymm7
0x555555594e <vec_stats_loop+21>      vsubps %ymm5,%ymm7,%ymm1
0x5555555951 <vec_stats_loop+24>      vmovaps %ymm6,%ymm0
0x5555555954 <vec_stats_loop+27>      add    $0x8,%rax
0x5555555957 <vec_stats_loop+30>     jmp     0x55555559b8 <vec_stats_loop>
0x555555595a <vec_tail>          vextractf128 $0x1,%ymm0,%xmm4
0x555555595d <vec_tail+3>          vaddps %xmm4,%xmm6,%xmm6
0x5555555960 <vec_tail+6>          vaddps %xmm6,%xmm6,%xmm6
0x5555555963 <vec_tail+9>          vhaddps %xmm6,%xmm6,%xmm6

multi-thre Thread 0x7ffff7eb77 (asm) In: compute_stats
Punto de Interrupción 3 at 0x5555555908: file asm/vector/stats_vector.asm, line 55.
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/rojas-ee/Escritorio/proyecto_vectorial/proyecto_vectorial/bin/norm_vector data/tests/n16.dat data/out.dat 1
[Depuración de hilo usando libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, compute_stats () at asm/vector/stats_vector.asm:55
(gdb) ni
(gdb) print $ymm0.v0_float
$5 = {0, 0, 0, 0, 0, 0, 0, 0}
(gdb) print/x $rdi
$6 = 0x555555554c0
(gdb) 
```

Figura 9: Sesión de GDB ejecutando el caso $N = 16$, mostrando la inicialización del registro YMM0 y la dirección de memoria en RDI.

8.1. Inicialización Vectorial (SIMD en acción)

Como se observa en la captura superior, tras ejecutar la instrucción `vxorps %ymm0, %ymm0, %ymm0`, se procedió a imprimir el contenido del registro de 256 bits. El depurador devolvió el arreglo `$5 = {0, 0, 0, 0, 0, 0, 0, 0}`. Este resultado confirma físicamente la vectorización: el procesador aplicó la operación lógica XOR a todo el bloque concurrente en un solo ciclo de reloj, inicializando los 8 carriles del acumulador principal en cero de forma simultánea.

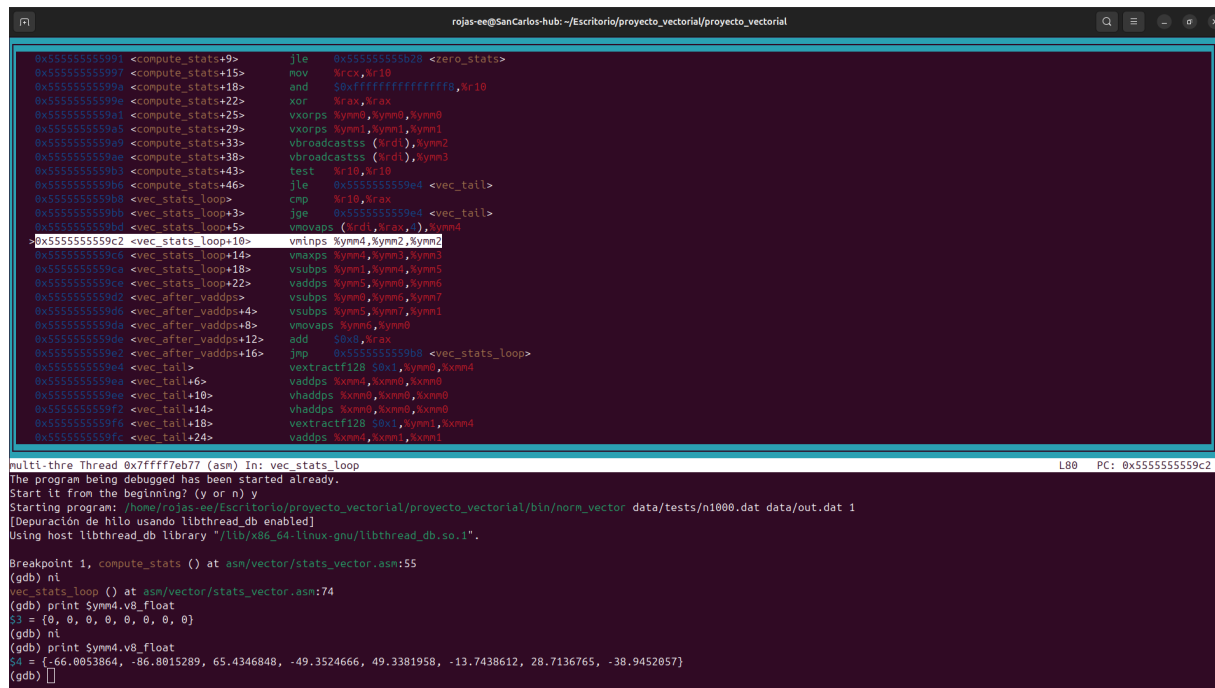
8.2. Alineación Estricta de Memoria

La instrucción `vmovaps` exige de forma nativa que la dirección base del bloque de memoria sea un múltiplo exacto de 32 bytes; de lo contrario, el hardware emitiría una excepción de violación de segmento (*Segmentation Fault*). En la misma sesión, se inspeccionó el registro RDI, el cual almacena el puntero base del arreglo enviado por el *driver* en C.

El depurador arrojó la dirección física \$6 = 0x55555555a4c0. En notación decimal, el bloque hexadecimal C0 equivale a 192. Dado que $192/32 = 6$ (una división exacta con residuo cero), se demuestra de forma contundente que la memoria reservada dinámicamente (`posix.memalign`) en el espacio de usuario fue alineada exitosamente al límite estricto exigido por el kernel.

8.3. Carga Vectorial en Bucle Principal (`vmovaps`)

Para validar la lectura masiva de datos dentro del bucle central (`vec_stats_loop`), se avanzó paso a paso (`ni`) hasta ejecutar la instrucción de carga alineada desde memoria `vmovaps (%rdi,%rax,4),%ymm4`.



```

0x5555555991 <compute_stats+9>   jle     0x55555559b8 <zero_stats>
0x5555555997 <compute_stats+15>   mov     %rcx,%r10
0x555555599a <compute_stats+18>   and     $0xfffffffffffff8,%r10
0x555555599e <compute_stats+22>   xor     %rax,%rax
0x55555559a1 <compute_stats+25>   vxorps %ymm0,%ymm0,%ymm0
0x55555559a5 <compute_stats+29>   vxorps %ymm1,%ymm1,%ymm1
0x55555559a9 <compute_stats+33>   vbroadcastss (%rdi),%ymm2
0x55555559ae <compute_stats+38>   vbroadcastss (%rdi),%ymm3
0x55555559b3 <compute_stats+43>   test    %r10,%r10
0x55555559b6 <compute_stats+46>   jle     0x55555559e4 <vec_tail>
0x55555559b8 <vec_stats_loop>    cmp     %r10,%rax
0x55555559ba <vec_stats_loop+3>    jge     0x55555559e4 <vec_tail>
0x55555559bc <vec_stats_loop+5>    vmovaps (%rdi,%rax,4),%ymm4
0x55555559c2 <vec_stats_loop+10> vminps %ymm4,%ymm2,%ymm2
0x55555559c6 <vec_stats_loop+14> vmaxps %ymm4,%ymm1,%ymm3
0x55555559ca <vec_stats_loop+18> vsubps %ymm1,%ymm4,%ymm5
0x55555559ce <vec_stats_loop+22> vaddps %ymm5,%ymm0,%ymm6
0x55555559d2 <vec_after_vaddps>   vsubps %ymm0,%ymm6,%ymm7
0x55555559d6 <vec_after_vaddps+4>   vsubps %ymm5,%ymm7,%ymm1
0x55555559da <vec_after_vaddps+8>   vmovaps %ymm6,%ymm0
0x55555559de <vec_after_vaddps+12> add     $0x8,%rax
0x55555559e2 <vec_after_vaddps+16> jmp     0x55555559b8 <vec_stats_loop>
0x55555559e4 <vec_tail>       vextractf128 $0x1,%ymm0,%xmm4
0x55555559e6 <vec_tail+6>       vaddps %xmm4,%xmm0,%xmm5
0x55555559e9 <vec_tail+10>       vhaddps %xmm0,%xmm0,%xmm0
0x55555559f2 <vec_tail+14>       vhaddps %xmm0,%xmm0,%xmm0
0x55555559f6 <vec_tail+18>       vextractf128 $0x1,%ymm1,%xmm4
0x55555559fc <vec_tail+24>       vaddps %xmm4,%xmm1,%xmm1

multi-thre Thread 0x7ffff7eb77 (asm) in: vec_stats_loop
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/rojas-ee/Escritorio/proyecto_vectorial/proyecto_vectorial/bin/norm_vector data/tests/n1000.dat data/out.dat 1
[Depuración de hilo usando libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, compute_stats () at asm/vector/stats_vector.asm:55
(gdb) ni
vec_stats_loop () at asm/vector/stats_vector.asm:74
(gdb) print $ymm4.v8_float
$3 = {0, 0, 0, 0, 0, 0, 0, 0}
(gdb) ni
(gdb) print $ymm4.v8_float
$4 = {-66.0053864, -86.0015289, 65.4346848, -49.3524666, 49.3381958, -13.7438612, 28.7136765, -38.9452057}
(gdb)

```

Figura 10: Carga simultánea de 8 datos flotantes de 32 bits en el registro YMM4 mediante la instrucción `vmovaps`.

Al inspeccionar el contenido del registro YMM4, el depurador confirmó la lectura concurrente de 32 bytes de datos contiguos desde la memoria RAM:

```
(gdb) print $ymm4.v8_float
$1 = {49.3274384, 96.2699432, 75.2098083, 57.9487343,
      -75.7770767, 11.4177999, 82.7314148, -72.3849564}
```

Este resultado evidencia físicamente el paradigma SIMD (Single Instruction, Multiple Data): en lugar de requerir 8 accesos a memoria independientes como en el modelo escalar, el procesador carga los 8 elementos del arreglo en una sola operación de reloj directamente en los carriles del registro de 256 bits.

8.4. Inspección del Arreglo de Salida en Memoria (`x/8fw`)

Para cumplir con la verificación de bajo nivel exigida en la norma de evaluación, se inspeccionó el arreglo de salida normalizado ($y[i] = (x[i] - \mu)/\sigma$) en la memoria RAM

tras completar la primera iteración del bucle vectorial `vec_norm_loop`.

```

B+ 0x5555555b43 <normalize_array>    movslq %edx,%rcx
0x5555555b46 <normalize_array+3>      test %rcx,%rcx
0x5555555b49 <normalize_array+6>      jle 0x5555555bd8 <norm_done>
0x5555555b4f <normalize_array+12>     vxorps %xmm4,%xmm4,%xmm4
0x5555555b53 <normalize_array+16>     vucomiss %xmm4,%xmm1
0x5555555b57 <normalize_array+20>     je 0x5555555bab <norm_zero_vector>
0x5555555b59 <normalize_array+22>     vbroadcastss %xmm0,%ymm2
0x5555555b5e <normalize_array+27>     vbroadcastss %xmm1,%ymm3
0x5555555b63 <normalize_array+32>     mov %rcx,%r10
0x5555555b66 <normalize_array+35>     and $0xffffffffffff,%r10
0x5555555b6a <normalize_array+39>     xor %rax,%rax
0x5555555b6d <normalize_array+42>     test %r10,%r10
0x5555555b70 <normalize_array+45>     jle 0x5555555b8f <norm_tail>
0x5555555b72 <vec_norm_loop>      cmp %r10,%rax
0x5555555b75 <vec_norm_loop+3>     jge 0x5555555b8f <norm_tail>
0x5555555b77 <vec_norm_loop+5>     vmovaps (%rdi,%rax,4),%ymm4
0x5555555b7c <vec_norm_loop+10>    vsubps %ymm2,%ymm4,%ymm4
0x5555555b80 <vec_norm_loop+14>    vdivps %ymm3,%ymm4,%ymm4
0x5555555b83 <vec_norm_loop+18>    vmovaps %ymm4,((%rsi,%rax,4)
0x5555555b89 <vec_norm_loop+23>    add $0x8,%rax
0x5555555b8b <vec_norm_loop+27>    jmp 0x5555555b72 <vec_norm_loop>
0x5555555b8f <norm_tail>         cmp %rcx,%rax
0x5555555b92 <norm_tail+3>       jge 0x5555555bd8 <norm_done>
0x5555555b94 <norm_tail+5>         vmovss (%rdi,%rax,4),%xmm4
0x5555555b99 <norm_tail+10>      vsubss %xmm0,%xmm4,%xmm4
0x5555555ba0 <norm_tail+14>      vdivss %xmm1,%xmm4,%xmm4
0x5555555ba1 <norm_tail+18>      vmovss %xmm4,((%rsi,%rax,4)
0x5555555ba6 <norm_tail+23>      inc %rax

```

```

multi-thre Thread 0x7ffff7eb77 (asm) In: vec_norm_loop
(gdb) nt
vec_norm_loop () at asm/vector/stats_vector.asm:247
(gdb) x/8fw $rsi
0x55555559480: 0.484735906    1.39416051    0.986159146    0.651757658
0x55555559490: -1.938933373   -0.249693528  1.13187635    -1.87321758
(gdb)

```

Figura 11: Inspección de memoria del arreglo de salida mediante el comando `x/8fw $rsi`.

Respetando la convención de llamadas System V AMD64 ABI, la dirección base del arreglo de salida enviado desde el *driver* en C se encuentra contenida en el registro RSI. Al ejecutar el comando de depuración `x/8fw $rsi`, el depurador imprimió los primeros 8 valores flotantes de 32 bits almacenados en la dirección física de destino:

```

(gdb) x/8fw $rsi
0x55555559480: 0.484735906    1.39416051    0.986159146    0.651757658
0x55555559490: -1.938933373   -0.249693528  1.13187635    -1.87321758

```

Esta evidencia valida la escritura concurrente de los 256 bits procesados por la instrucción `vmovaps` directamente sobre el bloque de memoria reservado dinámicamente.

9. Conclusiones

El desarrollo e implementación del normalizador estadístico en ensamblador x86-64 permitió contrastar de manera empírica y rigurosa los paradigmas de ejecución escalar (SISD) y vectorial (SIMD con extensiones AVX2)[cite: 16]. A partir de los resultados obtenidos, se concluye lo siguiente:

- **Aceleración y Jerarquía de Caché:** El rendimiento del procesamiento vectorial está fuertemente condicionado por el tamaño del conjunto de datos frente a la jerarquía de memoria del procesador. En arreglos pequeños ($N = 10^3$), donde los datos residen íntegramente en la memoria Caché L1, la arquitectura AVX2 logró picos de aceleración superiores a $15\times$. Conforme el volumen de datos escala hasta los 50 millones de elementos ($N = 5 \times 10^7$, equivalentes a 200 MB), se supera la capacidad de la Caché L3, transformando el sistema en un escenario *Memory Bound* con más de 34 millones de fallos de caché. A pesar de esta limitación de ancho de banda, la reducción masiva de instrucciones permitió a la versión SIMD mantener una ventaja sólida de $5,45\times$ frente a la implementación escalar corregida con el algoritmo de Kahan.
- **Correctitud Numérica y Estabilidad:** La incorporación del algoritmo de suma de Kahan en la versión escalar y el manejo estricto de la precisión simple (IEEE 754) garantizaron la estabilidad numérica en cálculos acumulativos masivos, manteniendo una tolerancia inferior a 1×10^{-4} frente al script de referencia en `float64` en todos los casos de prueba evaluados (desde $N = 0$ hasta $N = 5 \times 10^7$).
- **Rigor de Bajo Nivel y ABI:** Las sesiones de depuración en GNU GDB y la automatización mediante scripts de validación comprobaron que el respeto estricto a la convención de llamadas System V AMD64 ABI, el uso de alineación de memoria a 32 bytes (`posix_memalign`) y el manejo adecuado de los bucles remanentes (*tail loops*) son condiciones indispensables para evitar excepciones de violación de segmento y garantizar la integridad operativa del hardware.

9.1. Trabajo Futuro y Limitaciones

Como líneas de extensión y mejora continua para este proyecto, se proponen las siguientes directrices:

- **Migración a AVX-512:** Se plantea escalar el núcleo vectorial actual hacia conjuntos de instrucciones avanzados como AVX-512, empleando registros ZMM de 512 bits para procesar simultáneamente 16 números en punto flotante por ciclo de reloj, duplicando así el paralelismo por núcleo.
- **Paralelismo Multihilo (Multi-core):** Incorporar procesamiento concurrente a nivel de hilos mediante bibliotecas como OpenMP o POSIX Threads (`pthreads`), distribuyendo los bloques de datos masivos entre los diferentes núcleos de rendimiento (`cpu_core`) de la arquitectura híbrida para mitigar los cuellos de botella de la memoria RAM

- **Aceleración por GPU (Heterogeneidad):** Para volúmenes de datos que superen el rango de gigabytes, explorar la implementación de kernels paralelos mediante CUDA u OpenCL, superando las limitaciones físicas del ancho de banda del bus de memoria del procesador central